

On Overload Resolution, Exact Matches, and Clever Implementations

Document #: P3606R0 [[Latest](#)] [[Status](#)]
Date: 2025-01-24
Project: Programming Language C++
Audience: Evolution Working Group
Reply-to: Corentin Jabot
<corentin.jabot@gmail.com>
Younan Zhang
<zyn7109@gmail.com>

§ 1 Introduction

During overload resolution, if GCC finds an exact non-ambiguous, non-template match, it will pick that overload and will not perform template argument deduction.

```
template<typename T>  
decltype([] { return T::x; }) f(T); // #1  
void f(int) {} // #2  
  
int main() {  
    f(0);  
}
```

In this example, GCC will pick [#2](#) (that is an exact match for all arguments) when calling `f(0)`. GCC will not perform template argument deduction for [#1](#).

In other implementations, this is ill-formed.

Indeed, in other implementations, [#1](#) will be instantiated before overload resolution is performed, and `T::x` results in a substitution failure in a non-immediate context (body of a lambda), which is a hard error.

The conformity of GCC's behavior hinges on a generous interpretation of 13.9.2 [[temp.inst](#)]/9

“If the function selected by overload resolution can be determined without instantiating a class template definition, it is unspecified whether that instantiation actually takes place.”

Note that the [CWG consensus](#) is that GCC's behavior is probably valid by the spirit of the law but not its letter. In any case, CWG generally considers whether to mandate the GCC's behavior as an evolutionary matter.

Is it? After all, EWG might not care about whether diagnostics are produced in non-immediate contexts during overload resolution. Let's make things more riveting with concepts. Consider

```
struct S {  
    template <typename T>  
    requires std::copyable<T>  
    explicit S(T op) noexcept; // #1  
    S(const S&) noexcept = default; // #2  
};  
static_assert(std::copyable<S>);
```

This snippet is a reduction of code that is regularly submitted as an issue to implementations.

To check whether `S` satisfies `copyable`, we need to check that `S(declval<const S&>())` is a valid expression. To that end, we perform overload resolution between `#1` and `#2` after instantiating `#2` with `[T=S]`. `#1` is constrained by `copyable`.

To understand whether `S` satisfies `copyable`, one must first understand whether `S` satisfies `copyable`. The constraint depends on itself, and the program is ill-formed.

The user presumably wanted to write

```
template <typename T>  
requires (!std::same_as<T, S> && std::copyable<T>)  
explicit S(T op) noexcept;
```

But why didn't they? Probably because GCC is perfectly happy with this program. Indeed, When GCC performs overload resolution `#2` is a perfect match, `#1` is never instantiated, and its constraints are never checked.

Concept checking, in particular for concepts that could lead to recursion make GCC's optimization very much visible even without considering errors in non-immediate contexts.

Because of that, we think the standard needs to clarify what are the allowed implementation strategies. But this has been the status quo for years. Why write this paper now?

§ 1.1 CWG2369 makes the status quo more observable.

Clang tried to implement the very useful [\[CWG2369\]](#). However, this unearthed the opposite issue. Consider

```

template <typename U>
struct B { static_assert(false); };

template <typename T>
requires (sizeof(B<T>) == 1)
void f(T, typename T::foo = 0) {} // #1
void f(int) {} // #2

f(0);

```

Again, this code is reduced from actual bug reports. In particular, variation of that code have been found in the standard library specification, libc++, and MSSTL ([LWG4139]).

All compilers compile that code at the time of writing. As usual, GCC will pick #2 because it's an exact match, and #1 is, therefore, never instantiated.

Clang, however, will try to instantiate #1. Because [CWG2369] is not implemented in Clang, we will substitute into the argument `T::foo` **before** evaluating the constraint. Because `int::foo` is not a valid type, #1 SFINAE-away, and we pick #2.

Therefore, existing code happens to be valid for both GCC and Clang for different reasons, and that makes it difficult for Clang to deploy [CWG2369], even if that would be a very useful defect report to implement.

This brings us to this paper.

§ 2 Should the standard adopt GCC's behavior?

Adopting GCC's behavior, i.e., not considering template overload candidates when a non-template candidate is an exact match (i.e., there is an exact match for every argument) would have some benefits:

- The compiler has less work to do during overload resolution, improving compile times
- It would make the adoption of [CWG2369] less disruptive.
- It would save users who forget a `same_as` constraint on templated constructors. This pattern/mistake is fairly common (but then again, this only happens because an implementation is more lenient than others).

The downside is that errors in the non-immediate context of template candidates would no longer be diagnosed. A small price to pay?

On the other hand, if we do not want to bless GCC's behavior, it's unclear how implementations would be able to deploy CWG2369 as a DR, and should we expect GCC to change its implementation? That would break a non-trivial amount of code.

§ 3 Addendum: CWG2369 and recursive constraint checks.

Consider the following example [from GCC's bug tracker](#):

```
struct foo_tag{};
struct bar_tag{};

template <class T>
concept fooable = requires(T it) {
    invoke_tag(foo_tag{}, it);
};

template<class T>
T invoke_tag(foo_tag, T in); // #1

template<fooable T>
T invoke_tag(bar_tag, T it); // #2

int main() {
    // Neither line below compiles in GCC 11, independently of the other
    return invoke_tag(foo_tag{}, 2);
    return invoke_tag(bar_tag{}, 2);
}
```

In that scenario, both `invoke_tag` overloads are templates, so all implementations instantiate both overloads. After CWG2369, `fooable<T>` is checked for both overloads before looking at the validity of function arguments; that will cause the constraints of `#2` to be rechecked during satisfaction. `fooable<T>` always depends on itself and the program is ill-formed.

So GCC found that CWG2369 broke existing code (the above is a reduction from `tag_invoke`). To remediate this, they changed the order of template argument deduction to look at the arguments for which deduction would never produce an instantiation before considering constraints.

Doing so, in the example above, when checking the satisfaction of `fooable<T>`, GCC realizes `#2` is not a valid overload for `invoke_tag(foo_tag{}, it)`, and pick `#1`, which is not constrained, and therefore, there is no recursive checking, and both calls become valid.

No CWG issue was created for that, and it's unclear if it is EWG territory, but it is worth mentioning as GCC employed a similar strategy than they do for overload resolution to work around the breakages caused by CWG2369.

§ 4 Proposal Summary

During overload resolution, if there is a viable non-template candidate for which every argument's conversion is an exact match, then template candidate are not considered and

argument deduction/substitution does not happen.

§ 5 Implementation experience

All strategies described here were implemented by at least one implementation, which is exactly the issue!

§ 6 Wording

TBD.

Our main objective with this paper is to clarify intent so that we can facilitate the implementation of CWG2369 (and [CWG2769]) in Clang.

§ 7 Acknowledgments

Thanks to people who contributed to the various discussions on this topic on the Core reflector, and the LLVM and GCC bug trackers.

§ 8 References

[CWG2369] Agustin Bergé. 2017-10-09. Ordering between constraints and substitution.

<https://wg21.link/cwg2369>

[CWG2769] Richard Smith. 2023-07-14. Substitution into template parameters and default template arguments should be interleaved.

<https://wg21.link/cwg2769>

[LWG4139] Corentin. §[time.zone.leap] recursive constraint in <=>.

<https://wg21.link/lwg4139>